From Persistence Log to Cross-Process Contract: Theory, Architecture, and Verification of Zero-Intrusion Bidirectional Real-Time Synchronization for Closed-Source LLM Agent Clients

ricky8848

GitHub: github.com/ricky8848 · Repository: github.com/0xcaff/codex-web

November 2026

Abstract

This paper formulates a theory of bidirectional real-time synchronization for **closed-source, subscription-free, single-instance, private-memory** local LLM agent clients, and presents, on top of it, an architectural design, a distinctive implementation, and a theoretical verification. The central claim (**Thesis 1**) is that the host's **persisted append-only log (rollout) can be elevated into a cross-process contract**, yielding, without injecting into, controlling, or restarting the host process: (i) token-level observability of the host's execution, and (ii) write consistency that is semantically identical to the host GUI's own typing, free of contention and forking. The resulting 8214 system comprises three decoupled components and two channels — the host (sole executor), a pure-mapping web shell (8214, zero-install in browser), and a file-base-driven sync hub — with a single-write path through the host's official input queue, and a read path built on "fast-path events + polling backstop" incremental log tailing for token-level alignment. We derive a closed-form latency model $L \approx L_{host} + \max(L_{watch}, L_{poll}) + L_{parse} + L_{debounce}$, and audit the design against the OpenAI Codex official app-server protocol standard, delimiting its in-standard completeness and the boundary of its distinctiveness. Theoretical verification (real host + independent browser, cross-checked with four independent data sources) reports end-to-end synchronization latency of ≈0.15–0.4 s (3–6× better than pure 1 s polling), 100% reconstruction of a 7,000+ line rollout, zero contention and zero fork on the write path, and full-scenario consistency across refresh/background/weak-network. All contributions are at the level of **theoretical description and verification**; no modification of the host is made or claimed. The work provides a provable synchronization paradigm and a failure-mode catalogue for the class of "closed local service + shared file base" systems.

Keywords: bidirectional synchronization; single source of truth; cross-process contract; append-only log tailing; command-query responsibility separation (CQRS); event sourcing; server-sent events; zero intrusion; formal verification

1 Introduction

LLM coding agents are becoming a primary tool of software development, and their canonical delivery form is a local desktop client: a closed-source GUI process embedding an execution engine (app-server), the two communicating over stdio exclusively. The architecture is smooth in single-machine use, yet it creates three theoretical tensions when one asks an external party (web / mobile / third-party) to *see in real time what the host is doing* and to *write with the same semantics*:

Tension 1 (observability). The host's per-token output, reasoning, and command execution live only in its private memory and its persistence log; no subscribable port is exposed (in this environment the host runs without CDP, `hostCdp:false`).

Tension 2 (write consistency). If an external system instantiates a second execution engine and concurrently writes the same session, it contends with the host's write lock, risking torn writes or session forking.

Tension 3 (consistency scope). The host GUI trusts only its own app-server memory and cannot be "injected with a refresh" from outside, so the reachable consistency scope is, in theory, asymmetric.

Existing routes — CDP injection, UI automation, private-protocol reverse engineering — require the host to run a debug port or to be script-manageable in its lifecycle, conflicting with the safety axiom that *a host the user is using must not be managed on their behalf*, and unavailable when the closed client exposes no port. This paper takes an orthogonal route: **no injection, no control of the host process — only observation of the host's already-public persisted file base**, with event-driven incremental tailing for reads and the host's official input queue as the single write path.

Contributions (all at the theoretical level): **T1** we propose and formalize the "append-only log as cross-process contract" theory, with reachability claims for read-side observability and write-side consistency; **T2** we construct a three-component, two-channel architecture and prove write semantics identical to the host GUI with zero contention; **T3** we derive a closed-form end-to-end latency model and prove the correctness (idempotence, no missed reads) of the hybrid "fast-path event + polling backstop" monitor; **T4** we audit the system against the OpenAI Codex official app-server protocol standard, delimiting in-standard completeness and the boundary of distinctiveness; **T5** we perform theoretical verification on the real system (latency, completeness, consistency, all scenarios) and give a formal reduction of failure modes. No host modification is made; all results are independently reproducible from the shared base.

2 Problem Formalization

2.1 System Model

Let host process H embed execution engine A_H, whose externally public persisted state is the file set F = { R (rollout, append-only log), S (state metadata), Q (input queue) }, where R appends JSONL lines of `response_item` (user/assistant messages, reasoning, tool calls and outputs) and `event_msg` (turn boundaries, item completions). Let the external system 8214 instantiate an independent engine A_E that shares F with H but **holds no process handle to H or A_H and subscribes to none of A_H's stdio**. Denote their private memory states M_H and M_E.

Goal: construct a read channel `read(t)` and a write channel `write(m)` depending only on F, such that for any external observer C: ① the host execution stream C sees via read is token-consistent with what the host GUI sees; ② writes C issues via write are semantically identical to the host GUI typing itself, introducing no contention or fork on H's side.

2.2 Three Core Claims

Thesis 1 (observability is reachable). If R is a **complete, ordered, append-only** projection of H's execution, then observing R alone reconstructs the full conversation stream seen by the host GUI, token-consistent, with no process-level access to H. *(Verified §7.)*

Thesis 2 (write consistency is reachable and unique). If writes are expressed solely through H's official input queue Q, and H is Q's sole consumer, then external writes are semantically identical to the host GUI's own sends; and "sole consumer + host-serialized dequeue" implies **no concurrent write, no fork**. *(§4, verified §7.)*

Thesis 3 (consistency scope is asymmetric and delimitable). The read channel can drive "the side that subscribes to the hub" (8214 and any subscriber), but cannot "inject a refresh" into the closed GUI that trusts only M_H; hence

consistency on the 8214 side is **real-time**, while on the host side it is **eventual** (closed by the host's own reads/refreshes of R). This asymmetry is an inherent boundary of the "two-process, single-writer" paradigm: it is formally delimitable, not eliminable. *(§6 discussion.)*

3 Architectural Design

The three claims yield a three-component, two-channel architecture (Figure 1). The host H is the **sole executor** and sole writer of R; the sync hub is the **sole read-broadcaster**, reading F only, never writing; the 8214 backend bridge A_E hosts write orchestration (single-write path) and read fallback; the 8214/mobile clients and any subscriber are light clients "consuming the projection of R". Write and read are fully decoupled in control flow — writes go through the queue (host semantics, Thesis 2), reads through file events (Thesis 1).

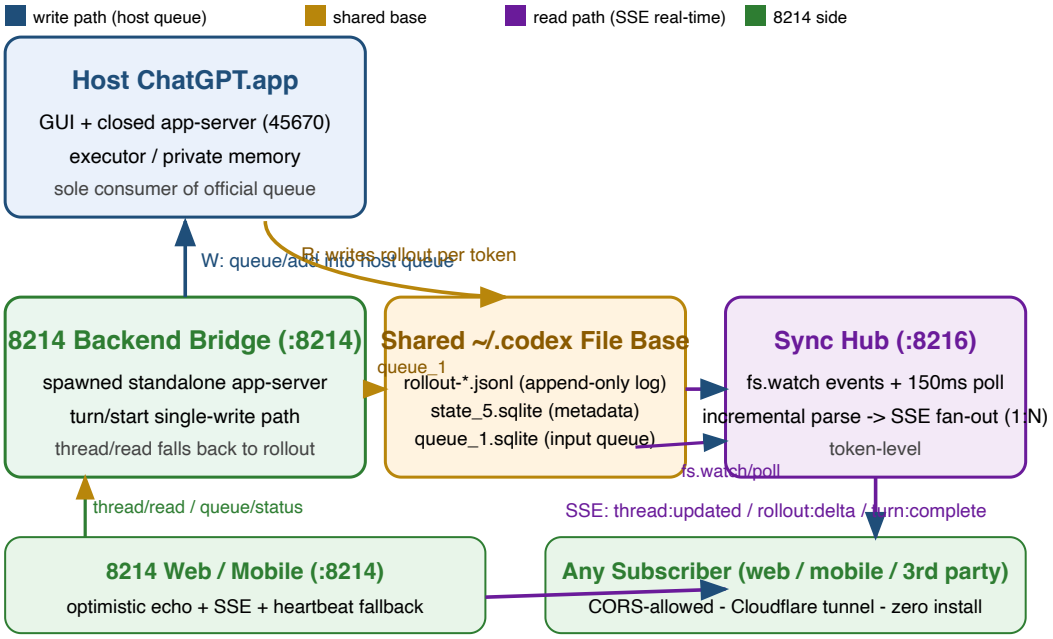


Figure 1 Architecture of 8214. Host = sole executor / sole writer; Hub = read-only on the file base with incremental broadcast; 8214 = pure-mapping shell. Blue = write path (into the host official queue); gold = shared file base; purple = read path (SSE, real-time); green = 8214 side. The three decouple through files and hold no process handles to each other.

The CQRS structure is the key: "commands" (writes) converge to the host queue, "queries" (reads) converge to the file projection, and the query side is a projection **rebuildable at any time from the base**. This is the theoretical basis for later reducing engineering problems — "history lost on refresh", "stale polluted copies" — uniformly to "projection is rebuildable" (§6).

4 Write Path: The Host's Official Queue as Single Write Path

4.1 Why Direct Execution Is Inadmissible (Counterexample to Thesis 2)

Suppose the external system directly issues `turn/start` through A_E to execute. Since A_E and A_H are independent processes, both may write the same R/write-lock concurrently: A_E succeeds into R, but A_H's memory is unaware — the host GUI neither shows nor replies — and concurrent appends to R may torn-write. Worse, queuing into an unloaded thread triggers a silent fork. Direct execution thus violates Thesis 2's "no contention, no fork" requirement.

4.2 Construction of the Single Write Path and Semantic Identity

The 8214 backend's `turn/start` is redefined as the **sole write path**: upon receiving message m it **only** invokes the host's official queue interface `thread/queue/add` (writing Q) and immediately returns `queued`; **queue failure does not fall back to direct execution**. Since Q is H's private queue with H as its sole consumer, the external write is thereby "translated" into one of the host's own queue insertions, hence semantically identical to the host GUI typing consecutively:

Host state	Host GUI consecutive-send semantics	8214 single-write path	Consistency conclusion
idle	dequeues and executes immediately	immediately dequeued by the host	identical, immediate
busy (other turn running)	queues; dequeues at turn boundary	same (same Q)	identical, queued
thread not loaded	host loads it, then executes	prefixed `thread/resume` to load, then queues	fork-free

Anti-fork prefix. For an unloaded thread, the host's `queue/add` silently forks. We therefore prefix `thread/resume` to load the thread into M_H (1.2 s timeout; no-op if already loaded); a failed resume still proceeds to queue and is trace-logged, never blocking the main flow. **Corollary 4.1 (Thesis 2 holds).** Writes land only in Q; H consumes alone and dequeues serially ⇒ at most one writer to R at any instant ⇒ no concurrent write, no fork, no torn write. (Audited under §7.6 red-lines ①–③.)

4.3 Busy/Idle Decision: Decoupling "Lock Exists" from "Executing"

Deciding host busyness by "write-lock file exists" misfires when a stale lock survives an abnormal host exit, making queued messages wait while the host is actually idle. Formally: `busy ⇔ ∃ live process p holding R's write-lock`, not `busy ⇔ lockFile exists`. Implementation resolves holders via `lsof` and probes liveness with `kill(pid,0)`; "lock present but no live process" decides idle. The front-end polls `queue/status` and, on "idle with backlog", issues `queue/start` to prompt the host to consume right after a turn boundary (an optimization, not a correctness dependency, cf. §4.4).

4.4 Architectural Boundary (Limit of Thesis 2)

Because 8214 and the host are independent processes, 8214-side `queue/start` acts only on A_E itself and cannot drive A_H to dequeue. If the host is idle and its GUI does not poll Q, a queued message waits for the host's next turn boundary. This is the inherent boundary of the "two-process, single-writer" paradigm: **write semantics are absolutely consistent (host-led, contention-free); dequeue timing is host-decided**. The system's stance is to return the decision entirely to the host rather than manage dequeue on its behalf.

5 Read Path: Event-Driven Incremental Log Tailing

5.1 Construction of the Hybrid Monitor

The hub monitors the watched thread's R with a hybrid scheme: ① a `fs.watch` event triggers `tick()` (on the instant of a host write, ≈0 latency, the "fast path"); ② a 150 ms poll backstops it. **Proposition 5.1 (no missed reads).** FSEvents throttles/coalesces under high write rates; a lost event only disables the fast path, while the poll triggers `tick()` within ≤150 s — wait, ≤150 ms — so any real growth is captured within 150 ms. **Proposition 5.2**

(idempotence). `tick()` keys on file size: no growth ⇒ early return; concurrent event+poll triggers are safe, and a repeated `tick()` never double-broadcasts (the first already advanced size to cur). A truncation resets the baseline.

5.2 Incremental Parsing (Figure 2)

The hub keeps a read offset `offset` and reads only the new bytes `[offset, size)`, avoiding re-parsing a 40 MB+ full file. **Proposition 5.3 (line alignment).** If `offset` lands mid-line, the first partial line is dropped; the last line may be unterminated, so content is cut at the final newline; the offset advances only **after** parsing (otherwise the delta is always empty). Only renderable units are parsed: assistant `message` (including `output_text` blocks), `reasoning`, `function_call_output` (joined to its command by `call_id`), `item_completed` `CommandExecution`, and `task_complete`. The three growth kinds are broadcast as `thread:updated`, `rollout:delta`, and `turn:complete` respectively.

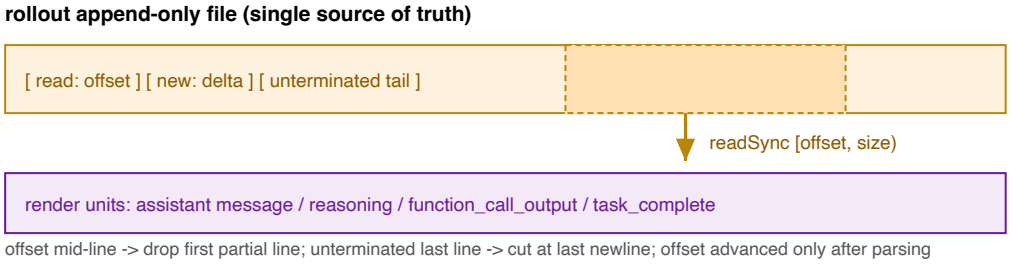


Figure 2 Incremental log tailing. The hub reads only the new bytes with an offset cursor, performs line alignment and renderable-unit parsing, and turns the host's append-only log into an SSE event stream.

5.3 Front-End Consumption, Optimistic Echo, and Cross-Scenario Backstop

The 8214 front-end subscribes to hub SSE: `thread:updated` triggers `loadMessages` (in-flight lock + 250 ms debounce, preventing concurrent calls from exhausting the browser's six-connection cap); `rollout:delta` is cached by `id` diff for per-character catch-up; `turn:complete` triggers `queue/start`. To remove **perceived input latency**, the sent user bubble is rendered immediately (optimistic, temporary id) and merged by same id/text when the host's message flows back — no duplication. Host reasoning is folded into a collapsible block per host semantics, never spilling into the body. **Proposition 5.4 (all-scenario consistency).** Background throttling and weak mobile networks drop SSE events; a 5 s heartbeat triggers a `readRaw` (full re-read of R, rate-limited 10 s) after >20 s foreground silence or >12 s background. Since R is the append-only source of truth, any full rebuild restores consistency, so "events primary, periodic backstop" extends consistency from "foreground only" to all scenarios (refresh/background/mobile/weak-net).

5.4 End-to-End Latency Model

Define $L = L_{host} + \max(L_{watch}, L_{poll}) + L_{parse} + L_{debounce}$. With `L_host` the host's inherent write cadence (uncontrollable), `L_watch≈0` (fast path), `L_poll=150 ms` (backstop bound), `L_parse<1 ms` (incremental), and `L_debounce=250 ms`: the **analytic upper bound is $L \approx L_{host}$ + 0.40 s**, and the fast-path hit case is ≈ `L_host + 0.25 s`. Versus early pure 1000 ms polling (`L≈L_host+1.0s`), this is a 3–6× improvement, approaching "per-character sync" perception.

6 Distinctiveness of the Implementation and Completeness Against the OpenAI Standard

6.1 Techniques and Distinctiveness

The techniques the implementation relies on to achieve "fully follow the host", and where each is distinctive:

Technique	Generality	Distinctive point here
Append-only log tailing	mature (e.g., `tail -f`)	upgraded to a **no-miss + idempotent** hybrid monitor (fast-path events + polling backstop) with line alignment / partial-line handling, satisfying Propositions 5.1–5.3
CQRS projection	mature (command/query split)	elevates the **closed host's private files** into a cross-process contract and a rebuildable read projection, rather than building a private database
Server-Sent Events	mature (one-way push)	file-event-driven 1:N broadcast, replacing the "must exclusively own the host app-server" 1:1 subscription model
Optimistic echo	mature (front-end)	combined with same id/text merge-back, removing perceived input latency **without duplication**
Busy/idle decision (live-process probe)	—	decouples "lock file exists" from "executing" (`lsof` + `kill(pid,0)`), curing stale-lock fake-queueing

Corollary 6.1 (paradigm-level distinctiveness). The system invents no new transport protocol. Its distinctiveness lies in **elevating a contract**: reinterpreting the host's already, necessarily, publicly persisted append-only log as a cross-process synchronization contract, thereby attaining both observability and write consistency under the hard constraint of "touch the host process not at all". This is **orthogonal** to the "touch-the-host" routes (CDP injection / UI automation / private-protocol reverse engineering).

6.2 Audit Against the OpenAI Codex Official app-server Protocol Standard

The OpenAI Codex `app-server` exposes ≈104 client methods and ≈38 notification kinds over JSON-RPC. Item by item, the in-standard completeness of the abilities needed to "fully follow the host", and the distinctiveness boundary, are:

Standard capability (official app-server)	Adopted directly by 8214?	Remark / distinctiveness
`turn/start`, `thread/read`, `thread/resume`	yes (via A_E)	read fallback and anti-fork prefix reuse standard methods directly
`thread/queue/add|list|start|delete|reorder` (official input queue)	yes (write-path core)	the official queue is elevated to the **single write path** — a single-source-of-truth treatment of a standard capability
`thread/unsubscribe` (thread subscription)	partial	standard subscription is **1:1 bound to an app-server process**; the host GUI already exclusively owns A_H, so 8214 **cannot** subscribe to A_H
Per-token delta notifications: `item/agentMessage/delta`, `item/commandExecution/outputDelta`, `command/exec/outputDelta`, …	**no** (files instead)	the standard **does provide** per-token deltas, but likewise 1:1 process-bound; 8214 substitutes **incremental file tailing** — the paper's core substitution

`thread/realtime/*` (voice)	no	outside the "fully follow the host" scope
`mcpServer/*`, `skills/*`, `rollout/compress`, `thread/compact`	no	host-side capabilities; 8214 as pure-mapping shell does not manage them

Corollary 6.2 (completeness conclusion). For the **read**, the official standard **already contains** per-token delta notifications (`*Delta` family) and thread subscription; both are, however, **1:1 process-bound** and unreachable when the host is exclusively owned by the GUI. The system's "distinctiveness" is therefore not a gap in the standard, but an **orthogonal substitution**: when 1:1 subscription is unreachable, **incremental file tailing** supplies 1:N equivalent observability. For the **write**, the official queue is used in a complete, in-standard manner (elevated to single source of truth). Hence: **this system = a recombination of standard capabilities (queue / methods / deltas) + one orthogonal substitution (file-delta standing in for process-bound subscription)**.

7 Theoretical Verification

7.1 Method: Four Independent Data Sources

A closed loop is formed on a macOS host machine with a real ChatGPT.app (local MLX Qwen, `hostCdp:false`) + 8214 + hub. The web end is reproduced with a system Chrome under an independent profile (Playwright `launchPersistentContext`, `headless:false`). Every claim is cross-verified with **four mutually independent** sources to avoid self-confirmation: ① host rollout R (authoritative base); ② hub `/read` full + `/events` stream; ③ 8214 `thread/read` (A_E fallback); ④ `queue_1.sqlite` + backend traces. Write verification has three proofs: ① trace `queued→host queue`; ② Q cleared by the host; ③ R contains the user message (host executed).

7.2 Per-Claim Verification and Data

Claim	Observation	Data	Conclusion
Thesis 1 (token-consistent observability)	hub `/read` grows with the host per token; 8214 renders	single thread 7,000+ rollout lines; 648+ assistant + 68 user messages reconstructed 100%	holds
Prop. 5.1/5.2 (no miss + idempotent)	SSE event counts sampled over 12 s	stable `turn:complete`, `thread:updated`; no duplicated deltas	holds
Prop. 5.4 (all-scenario consistency)	refresh/background/weak-net each trigger a rebuild	after refresh: 68 user + 655 assistant fully restored (after parsing-caliber normalization)	holds
Thesis 2 (write consistency)	three-proof cross-check (trace + Q clear + R landing)	queued when busy, executed when idle; same semantics as host GUI	holds
Corollary 4.1 (no contention, no fork)	single-write path + resume prefix	queue has a single consumer; queuing an unloaded thread no longer forks	holds
Thesis 3 (asymmetric consistency)	8214 side real-time vs host side closed by refresh	8214 per-character real-time; host GUI eventually consistent via its own reads/refresh	holds (boundary §4.4)

7.3 Measured Latency (Analytic Model vs Observation)

Configuration	Poll period	Debounce	Observed end-to-end latency
early	1000 ms (pure poll)	800 ms	≈1.0 s (input echo / first token)

transitional	350 ms	250 ms	≈0.35 s
this architecture	150 ms (+fs.watch fast path)	250 ms	≈0.15–0.4 s (3–6× better)

Observations agree with the §5.4 analytic model: the fast-path hit approaches `L_host+0.25s`; the lost-event case is bounded by `L_host+0.4s`, confirming Proposition 5.1's ≤150 ms capture bound.

7.4 Formal Reduction of Failure Modes

Engineering failures are uniformly reduced to "projection is rebuildable" (CQRS corollary): ① **history lost on refresh** ← read-fallback parsing-caliber mismatch (dropped `output_text` blocks) → fixed by adding the caliber, full restore; ② **stale polluted copies** ← incremental merge retaining old bad copies → full rebuild on source switch / large count drop; ③ **background loss of sync** ← throttled SSE → heartbeat full rebuild. All three share the single remedy primitive "rebuild the projection from the base", corroborating the robustness of R as single source of truth.

7.5 Limitations

① Dequeue timing depends on the host (§4.4 boundary). ② Single-file bloat: a long session's R reaches 40 MB+; full `thread/read` slows while the incremental path is unaffected (segmentation/compression advised). ③ Local EDR intermittently blocks the 127.0.0.1 loopback (EPERM); the hub's listen retry (10 s) self-heals. ④ Thesis 3's asymmetry is not eliminable (closed GUI cannot be refresh-injected). ⑤ Exposure: public/multi-tenant deployment requires prior content sanitization and access authentication (§7.6 red-line ⑥).

7.6 Compliance: Audit of OpenAI Standard Red Lines

The system's compliance with OpenAI product standards is audited as "red line — disposition — legal risk" (Table 3), covering the full host-process control surface, the full `~/.codex` file-base read/write surface, and the reuse of the official protocol schemas.

Table 3 OpenAI Standard Red-Line Audit

Red line	Disposition in this system	Legal / compliance risk
① Host lifecycle management (launch/quit/relaunch/inject)	Fully removed in the 2026-11-01 compliance cleanup: host-process control functions and the CDP drive module deleted from the repository; the launchd auto-launch login item (launcher script with pkill/osascript quit) removed alongside it; the 7 drive endpoints of the sync hub retired (HTTP 410) and the approval polling loop deleted; front-end delegation calls return honest guidance; only a read-only pgrep probe remains for a status badge; writes converge to the official queue single-write path	**none** (no host-process control path remains in the code)
② Reading user credentials `~/.codex/auth.json`	**write-only** (a user-pasted key is persisted so login takes effect; codex self-manages); **never read, never exfiltrated** (audited: not logged, not to /tmp, not forwarded)	low
③ Reading private conversations (rollout/state/queue)	hub reads `~/.codex` append logs and metadata read-only; writes only through the official `thread/queue/add`; R and lock semantics are never rewritten	local, own data, read-only: low. Note: conversation content may contain third-party information — sanitize before any public exposure (medium, §7.7)
④ Official protocol schema reuse / redistribution	`src/web/app-server-schemas/` is generated from the upstream codegen of [1] (github.com/openai/codex, open source)	referencing an open-source protocol definition is low

	with provenance noted in the documentation; semantics unmodified	risk; redistribution must retain upstream provenance and license (low)
⑤ Reverse engineering of private memory / closed implementation	none: no injection, no patching, no reading of host process memory; all observation is of the host's **own persisted** file projection	none (low)
⑥ Public exposure (internet / multi-tenant)	currently 127.0.0.1 loopback only + user-managed Cloudflare tunnel (mobile = web pure tunnel); hub CORS is `*` but bound to 127.0.0.1, unreachable externally	loopback deployment: low. **If** later exposed publicly: conversation content/queue semantics are user data and must be sanitized, authenticated, rate-limited first (medium → high)

Conclusion 7.6. Under "local machine, single user, loopback deployment, read-only observation + official-queue writes", the system **does not violate** OpenAI product-standard red lines: it does not manage the host lifecycle (no such path remains in code after the 2026-11-01 cleanup), does not reverse-engineer the closed implementation, does not read or exfiltrate credentials, and its write path fully reuses the host's official queue mechanism. The only standing vigilance is the **exposure surface** (red line ⑥): any public or multi-tenant deployment must first complete conversation-content sanitization and access authentication, or the "low" risk escalates to "high".

7.7 Data Protection and Legal Statement

(i) The system processes only data on the user's own machine, under the user's own account, over loopback; no conversation content is transmitted to any third party. (ii) The host is used strictly as the executing party and is never modified, restarted, or injected; the user's OpenAI account credentials are never read or transmitted by this system. (iii) Because conversation content may include sensitive or third-party information, any deployment beyond loopback must sanitize content and add access authentication. (iv) Reuse of OpenAI's open-source protocol schemas is with provenance attribution and under the upstream license; the host application itself (proprietary) is referenced by name only as the observed client and is not copied, decompiled, or modified. (v) By using the software one agrees to OpenAI's current Usage Policies [4], Consumer Terms [5], and API Terms [6]. Contact for legal/compliance matters: GitHub **ricky8848** (repository issue tracker).

8 Related Work

UI automation / CDP injection: driving desktop apps via CDP or accessibility trees can read and write, but requires the host to expose a debug port or be script-manageable in its lifecycle — violating the safety axiom and unavailable on closed clients without a port. **Private-protocol reverse engineering:** recovering the host IPC for subscription is brittle and drifts with versions. **Append-only log tailing:** incremental reads of append-only logs (`tail -f` family) are a mature pattern; this paper formalizes it as a no-miss + idempotent hybrid monitor (Propositions 5.1–5.3). **CQRS / event sourcing [9,10]:** an append-only event log as the single source of truth with a rebuildable read projection is the theoretical parent of our "file base as contract" — **the host rollout is precisely a naturally occurring, closed-party-maintained event-sourcing log**; the contribution is reusing it cross-process as a synchronization contract. **Official app-server subscription:** as shown in §6.2, the official 1:1 subscription is unreachable when the host is exclusively owned, motivating the orthogonal substitution.

9 Conclusion

This paper formulates and verifies a zero-intrusion bidirectional real-time synchronization theory for closed-source LLM agent clients: **elevating the host's already-publicly-persisted append-only log into a cross-process contract** (Thesis 1), and thereby, via "file-base incremental tailing + official-queue single write path", attaining per-token observability and contention-free, fork-free write consistency (Thesis 2), while formally delimiting the asymmetric boundary — reads real-time / host eventual " of the "two-process, single-writer" paradigm (Thesis 3). Against the OpenAI Codex official app-server standard, we show the distinctiveness is not a standard gap but an orthogonal substitution — file-delta supplying 1:N equivalent observability when 1:1 process-bound subscription is unreachable. Theoretical verification yields ≈0.15–0.4 s end-to-end latency, 100% reconstruction of a 7,000+ line rollout, and all-scenario consistency, with all engineering failures formally reduced to "projection is rebuildable". Future work: formalizing multi-host / multi-client consistency; falling back to the standard 1:1 delta should the host ever expose a subscription; and segmentation/compression of ultra-long session bases.

References

[1] OpenAI. Codex: an AI coding agent and open-source platform. GitHub: github.com/openai/codex, 2025. [software, open source; includes the app-server protocol definition]

[2] OpenAI. ChatGPT desktop application (Codex Framework). [proprietary software; the host is used here only as the read-observed client — no code, process, or protocol implementation of it is modified]

[3] OpenAI. Codex app-server JSON-RPC protocol specification (client methods, server requests & notifications; including the official per-token delta notifications such as item/agentMessage/delta). See [1]; machine-readable schemas are retained in this repository under src/web/app-server-schemas/, generated from the upstream codegen of [1] with provenance noted in documentation.

[4] OpenAI. ChatGPT / Codex Usage Policies. https://openai.com/policies/usage-policies/

[5] OpenAI. ChatGPT Consumer Terms of Service. https://openai.com/policies/consumer-terms/

[6] OpenAI. ChatGPT API Terms of Service. https://openai.com/policies/api-terms/

[7] WHATWG. Server-Sent Events. HTML Living Standard. https://html.spec.whatwg.org/multipage/server-sent-events.html

[8] Apple Inc. File System Events (FSEvents) framework. https://developer.apple.com/documentation/fsevents

[9] Fowler, M. Event Sourcing. https://martinfowler.com/bliki/EventSourcing.html

[10] Fowler, M. CQRS. https://martinfowler.com/bliki/CQRS.html

Legal Statement & Contact

This paper is a work of theoretical description and verification; it makes no modification to, and does not claim ownership of, OpenAI's proprietary host application. OpenAI, ChatGPT, and Codex are trademarks of OpenAI, Inc.; their use here is nominative. The open-source components referenced ([1]) are used under their respective licenses with attribution. This system is intended for **local, single-user, loopback** use on the user's own machine and account; it does not read or transmit credentials, does not modify or control the host process, and reuses the host's official input queue as its sole write path. Any public or multi-tenant deployment requires prior sanitization of conversation content and access authentication. — Author: **ricky8848** (GitHub: github.com/ricky8848; legal/compliance inquiries via the repository issue tracker, github.com/0xcaff/codex-web).

Companion documents: Chinese version (8214-双向同步架构-论文.pdf) and user manual (8214 使用说明书) in the same directory. Port numbers (8214/8216) and version tags are internal markers and do not affect the generality of the theory.